Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

За шестой семестр

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Бинарная классификация с использованием кросс-энтропии (логистическая регрессия)»

Выполнил:
Студент 3 курса
Группы ИИ-26(2)
Рулько М.А.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провести **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Задачи лабораторной работы:

1. Модифицировать программу из ЛР №1:

- Заменить пороговую функцию активации на сигмоидную.
- Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
- Реализовать вычисление градиента ŷ-exi для обновления весов.

2. Реализовать обучение с **фиксированным** шагом (взять значение α, показавшее наилучший результат в ЛР №1).

Запоминать суммарную ошибку Esp на каждой эпохе p.

3. Реализовать обучение с **адаптивным** шагом:

- На каждой итерации (после подачи одного образа) вычислять t (формула (2.36)).
- Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
- Сохранять значения Esp для построения графика.

4. Провести сравнительный анализ:
    - Построить на одном графике кривые сходимости Esp для четырёх конфигураций:
        - MSE + фиксированный шаг (данные из ЛР №1);
        - MSE + адаптивный шаг (данные из ЛР №2);
        - BCE + фиксированный шаг;
        - BCE + адаптивный шаг.
    - Зафиксировать число эпох, необходимое для достижения заданной точности Es≤Ee (взять Ee из ЛР №2, например 0.01).

5. Визуализировать результаты:
    - Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
    - Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
    - Для сравнения также показать линию из ЛР №1 (если она отличается). **Описать** сравнение двух линий.

6. Реализовать режим функционирования:
    - Пользователь вводит координаты x1,x2.
    - Сеть вычисляет вероятность ŷ и, применяя порог, определяет предсказанный класс.
    - Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).

7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE. Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.

8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

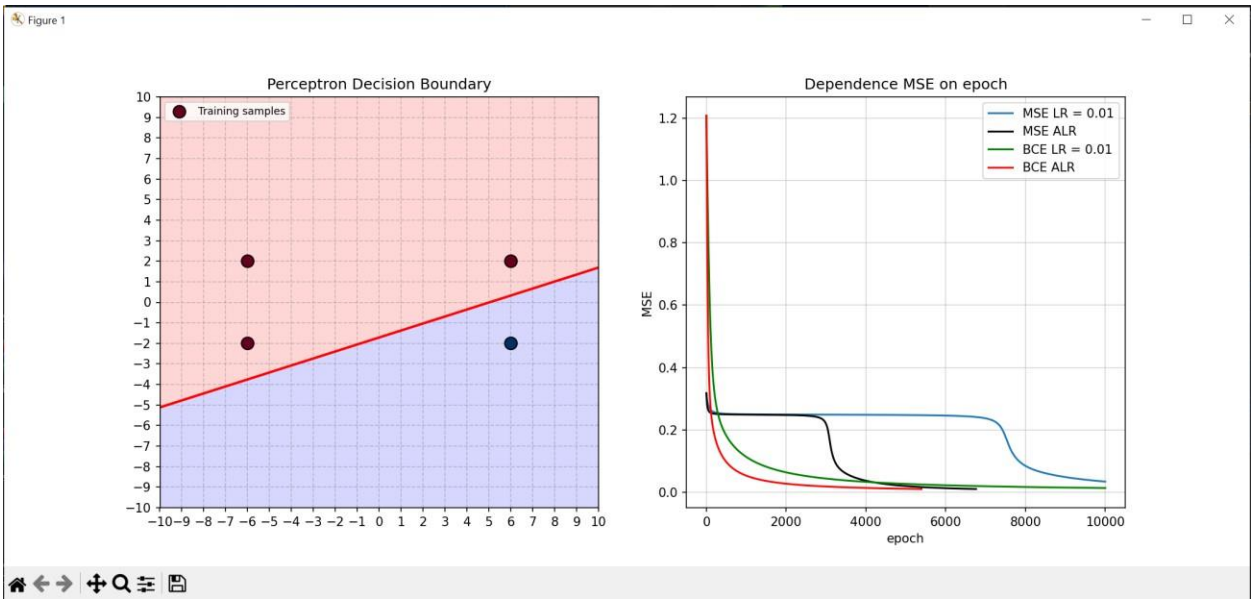
Код программы:

```
defmodule LogisticRegressor do
  @moduledoc
  defstruct [:w1, :w2, :bias, :lr, :target_accuracy]

  def new(lr \\ 0.1, target_accuracy \\ 0.01) do
    %LogisticRegressor{
      w1: :rand.uniform() * 0.2 - 0.1,
      w2: :rand.uniform() * 0.2 - 0.1,
      bias: :rand.uniform() * 0.2 - 0.1,
      lr: lr,
      target_accuracy: target_accuracy
    }
  end

  # SIGMOID
  def sigmoid(z), do: 1.0 / (1.0 + :math.exp(-z))

  # z=w1*x1+w2*x2+bias*(-1.0)
  def predict_proba(model, x1, x2) do
    z = model.w1 * x1 + model.w2 * x2 + model.bias * -1.0
    sigmoid(z)
  end

  # Enum.reduce — это замена циклу for p in dataset sum +=
  def calculate_bce(model, dataset) do
    # защита от log0 чтобы программа не упала
    eps = 1.0e-15
    n = length(dataset)

    sum =
      Enum.reduce(dataset, 0.0, fn p, acc ->
        y_hat = predict_proba(model, p.x1, p.x2)
        y_hat_safe = max(eps, min(1.0 - eps, y_hat))
        acc + (p.label * :math.log(y_hat_safe) + (1.0 - p.label) * :math.log(1.0 - y_hat_safe))
      end)

    # возвращаем среднее значение
    -(sum / n)
  end

  # Одна эпоха: полный проход по данным с корректировкой весов.
  def train_epoch(model, dataset, use_adaptive \\ false) do
    # Расчет шага обучения (Learning Rate).
    # Если включен Adaptive Step, считаем 1 / среднюю энергию сигналов.
    current_lr =
      if use_adaptive do
        avg_energy =
          Enum.reduce(dataset, 0.0, fn p, acc ->
            acc + (:math.pow(p.x1, 2) + :math.pow(p.x2, 2) + :math.pow(-1.0, 2))
          end) / length(dataset)

        1.0 / avg_energy
      else
```

```
      model.lr
    end

  # ГРАДИЕНТНЫЙ СПУСК:
  # В Elixir переменные неизменяемы, поэтому Enum.reduce здесь работает так:
  # Берем модель -> применяем одну точку данных -> получаем НОВУЮ модель ->
  # -> передаем её на следующую точку. В конце получаем финальную версию за эпоху.
  new_model =
    Enum.reduce(dataset, model, fn p, %LogisticRegressor{} = m ->
      y_hat = predict_proba(m, p.x1, p.x2)
      # Направление и величина ошибки
      error = y_hat - p.label

      # Синтаксис %{ m | ... } создает копию структуры 'm' с новыми значениями.
      %{
        m
        | w1: m.w1 - current_lr * error * p.x1,
          w2: m.w2 - current_lr * error * p.x2,
          bias: m.bias - current_lr * error * -1.0
      }
    end)

  # Возвращаем кортеж обновленная_модель ошибка_на_этой_эпохе
  {new_model, calculate_bce(new_model, dataset)}
 end

 # В Elixir НЕТ WHILE ТАК ЧТО МЫ ИСПОЛЬЗУЕМ РЕКУРСИЮ ДЛЯ ОБУЧЕНИЯ
 def train(model, dataset, use_adaptive, epoch, history) do
  # Выполняем шаг обучения
  {updated_model, bce} = train_epoch(model, dataset, use_adaptive)

  # Вывод в консоль каждые 100 итераций
  if rem(epoch, 100) == 0 do
    IO.puts("Epoch #{epoch}: BCE = #{Float.round(bce, 6)}")
  end

  if bce <= model.target_accuracy or epoch >= 5000 do
    {updated_model, Enum.reverse([bce | history])}
  else
    train(updated_model, dataset, use_adaptive, epoch + 1, [bce | history])
  end
 end
end

dataset = [
 %{x1: 6, x2: 2, label: 0},
 %{x1: -6, x2: 2, label: 0},
 %{x1: 6, x2: -2, label: 1},
 %{x1: -6, x2: -2, label: 0}
]

initial_model = LogisticRegressor.new(0.1, 0.01)

IO.puts("Starting training (Adaptive Step)...")
{final_model, _history} = LogisticRegressor.train(initial_model, dataset, true, 1, [])
```

```
IO.puts("\nFinal Weights:")
IO.inspect(final_model)

test_point = {6.0, -2.0}
prob = LogisticRegressor.predict_proba(final_model, elem(test_point, 0), elem(test_point, 1))
class = if prob >= 0.5, do: 1, else: 0

IO.puts("\nTest Point #{inspect(test_point)}:")
IO.puts("Probability: #{Float.round(prob, 4)}")
IO.puts("Predicted Class: #{class}")
```

Результат:



Как мы можем видеть BCE справляется лучше MSE на сигмоидальной функции. На графиках MSE мы видим горб/ступеньку – он образуется из-за плато образующегося при использовании сигмоидной функции (при S = 0 y = 0.5 – здесь у нее низкая чувствительность из-за производной), как только шаг преодолевает ее, происходит резкий скачок. В BCE же производная не используется поэтому таких плато нет

Вывод: Изучил принципы бинарной классификации с вероятностной точки зрения. Реализовал однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провел **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).